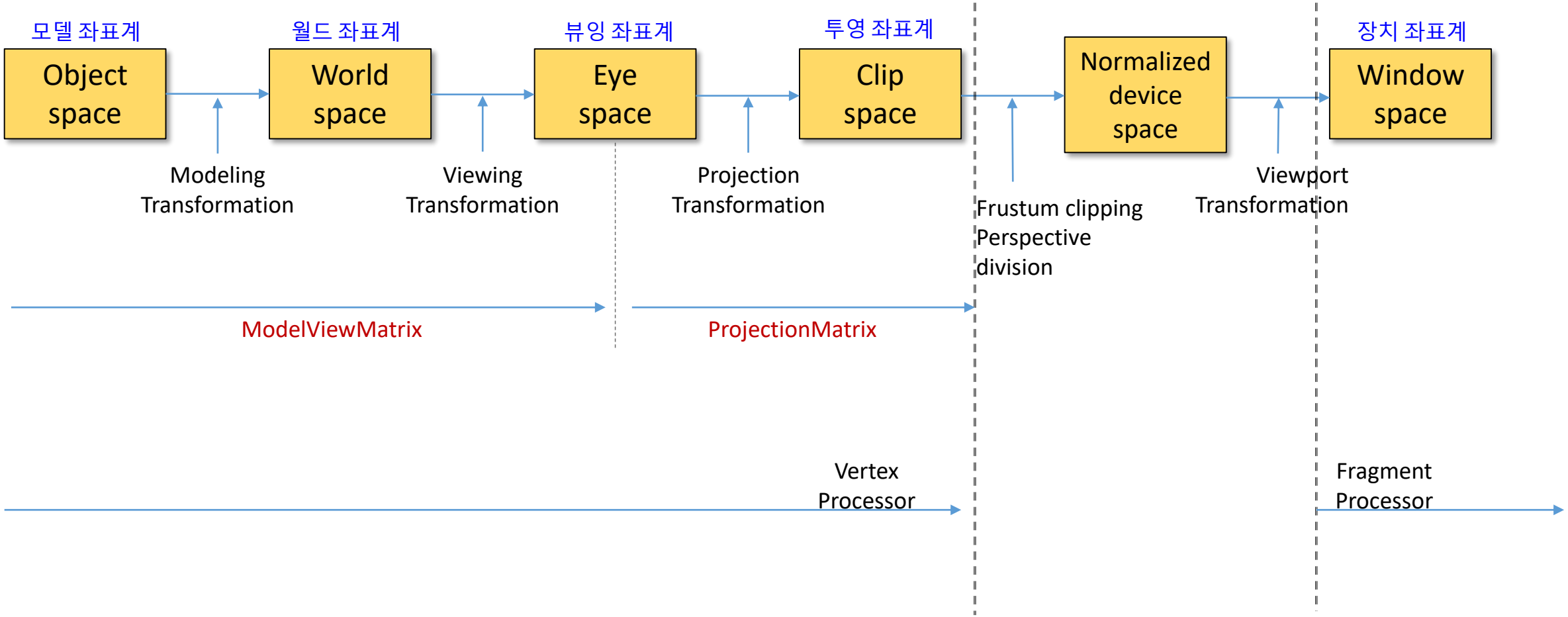
openGL 좌표계 변환 2

2026년 2학기

3차원 좌표계

- 좌표계 변환
  - 물체는 좌표계에 따라 새로운 좌표값으로 바뀌어 최종적으로 화면에 그려진다.



2) 뷰잉 변환

- 뷰잉 변환: 월드 좌표계를 유저의 시점인 view space (뷰 공간)로 변환
  - View space: 월드 공간의 객체들을 **카메라를 원점으로 하는 좌표계**로 변환한 공간
    - 즉, 카메라의 관점에서 바라보는 좌표 공간
    - 기본 카메라 좌표계
      - 카메라는 원점 (0, 0, 0)위치
      - 오른쪽 방향: +x축
      - 위쪽 방향: +y축
      - 바라보는 방향: -z축

  - 카메라 위치 지정 방법

    1) 모델링 변환 함수들을 사용하여 카메라의 위치를 바꿀 수 있다
       - 카메라의 위치 변환은 객체들의 위치 변환과 반대로 변환하는 것과 같다.
         - 예를 들어, 카메라를 오른쪽으로 이동 = 객체를 왼쪽으로 이동
       - 연속된 회전과 이동으로 카메라 위치를 지정할 수 있다.

    2) 카메라 설정 함수로 카메라 위치를 바꿀 수 있다.

2) 뷰잉 변환

- 뷰잉 변환 적용하기
  - 뷰잉 변환 행렬 **M_{view} 행렬** 값에 변환 함수를 적용하여 카메라의 위치를 바꾼 후, 버텍스 세이더에 적용한다.

```
//--- 응용 프로그램
void drawScene ()
{
	glUseProgram (shaderProgram);
	glm::mat4 view = glm::mat4(1.0f);
	view = glm::translate (view, glm::vec3(0.0f, 0.0f, -3.0f));        //--- 월드의 모든 객체를 z축으로 -3만큼 이동시키는 뷰 변환
                                                                       //--- ➔ 카메라를 +z축으로 3만큼 이동한 효과
	unsigned int viewLocation = glGetUniformLocation (shaderProgram, "viewTransform"); //--- 버텍스 세이더에서 viewTransform  변수위치
	glUniformMatrix4fv (viewLocation, 1, GL_FALSE, &view[0][0]);       //--- viewTransform 변수에 변환값 적용하기

	glBindVertexArray (VAO);
	glDrawArrays (GL_TRIANGLES, 0, 3);
}
```

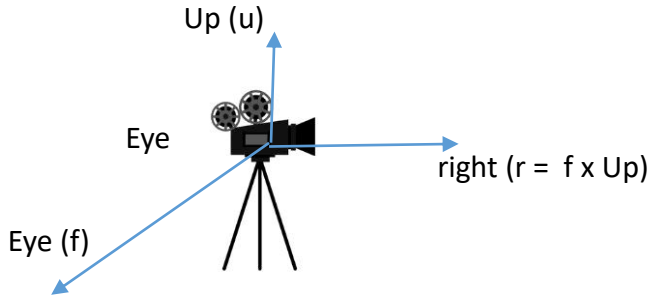
```
//--- 버텍스 세이더
#version 330 core
layout (location = 0) in vec3 vPos;

uniform mat4 modelTransform;
uniform mat4 viewTransform;

void main()
{
		gl_Position = viewTransform * modelTransform * vec4(vPos, 1.0);
}
```

2) 뷰잉 변환

- 카메라 설정
  - 다음의 세 가지 파라미터를 가진다.
    - Eye: 월드 공간에서의 카메라의 위치 → (e_x, e_y, e_z)
    - At: 월드 공간에서 카메라가 바라보는 목표점 → (a_x, a_y, a_z)
    - Up: 카메라의 위쪽 방향 → (u_x, u_y, u_z) (대부분의 경우 Up은 월드 공간의 y축)



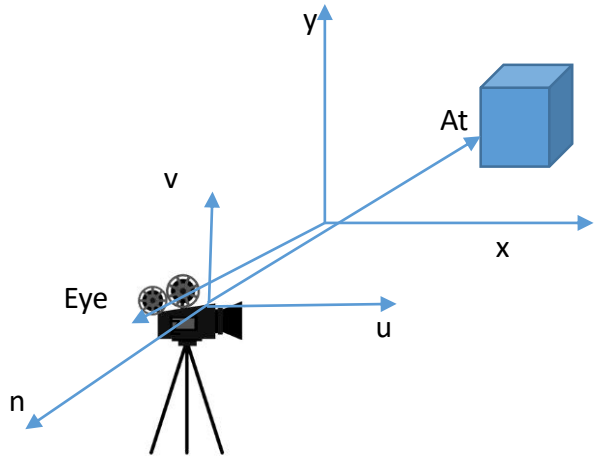
  - 벡터 설정
    - Forward (z축): 카메라가 바라보는 방향 → $f = \frac{At - Eye}{||At - Eye||}$
    - Right (x축): 카메라의 오른쪽 방향 → $r = \frac{f \times Up}{||f \times Up||}$
    - Up (y축); 카메라의 실제 위쪽 방향 → u = r x f

  - 카메라 공간 설정을 위한 3차원 뷰잉 변환 행렬은
    - $T = \begin{bmatrix} 1 & 0 & 0 & -e_x \\ 0 & 1 & 0 & -e_y \\ 0 & 0 & 1 & -e_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$ $R = \begin{bmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ -f_x & -fy & -f_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ $M_{view} = R \cdot T = \begin{bmatrix} r_x & r_y & r_z & -r \cdot e \\ u_x & u_y & u_z & -u \cdot e \\ -fx & -fy & -f_z & -f \cdot e \\ 0 & 0 & 0 & 1 \end{bmatrix}$

r: 오른쪽 벡터
u: 위쪽 벡터
f: 방향 벡터
e: 카메라의 위치 벡터

2) 뷰잉 변환

- 카메라 공간
  - 다음의 세 가지 파라미터를 가진다.
    - Eye: 월드 공간에서의 카메라의 위치
    - At: 월드 공간에서 카메라가 바라보는 기준점
    - Up: 카메라의 위쪽 방향을 결정하기 위한 기준 벡터
      - 대부분의 경우 Up은 월드 공간의 y축
    - 단위 벡터와 외적 등을 이용하여 카메라 공간의 좌표축을 설정한다.



  - 카메라 공간의 좌표축 설정
    - 카메라 위치:
      - glm::vec3 cameraPos = glm::vec3 (0.0f, 0.0f, 5.0f); //--- 카메라 위치
    - 카메라 기준점
      - glm::vec3 cameraTarget (0.0f, 0.0f, 0.0f);
    - 기준 Up 벡터
      - glm::vec3 worldUp (0.0f, 1.0f, 0.0f); //--- 대부분 월드 공간의 y축을 사용한다.
    - 카메라가 바라보는 방향의 반대 방향 (n) → 카메라 좌표계의 z축
      - glm::vec3 **cameraDirection** = glm::normalize (cameraPos-cameraTarget); //--- 카메라가 바라보는 방향의 반대 방향:+z
    - 카메라의 오른쪽 축 (u) → 카메라 좌표계의 x축
      - 기준 Up 벡터와 카메라 방향 벡터와의 외적
      - glm::vec3 **cameraRight** = glm::normalize (glm::cross (worldUp, cameraDirection)); //--- 카메라의 오른쪽 방향: +x
    - 최종적으로 계산된 카메라 좌표계의 위쪽 축 (v) → 카메라 좌표계의 y축
      - glm::vec3 **cameraUp** = glm::normalize(glm::cross(cameraDirection, cameraRight)); //--- 카메라의 위쪽 방향: +y

2) 뷰잉 변환

- 카메라 설정 함수 프로토타입
  - glm::mat4 glm::**lookAt** (vec3 const &cameraPos, vec3 const &cameraTarget, vec3 const &cameraUp);
    - cameraPos: 카메라의 위치
    - cameraTarget: 카메라가 바라보는 기준점
    - cameraUp: 카메라의 상단이 가리키는 방향

  - 카메라 변환 위해서는
    - 위의 세 인자값 변경
    - 변환 함수를 적용하여 카메라 위치 및 방향 변경 가능

  - 사용 예)
    - 카메라를 (0, 0, 3) 위치에 두고 원점 (0, 0, 0)을 바라보게 한다.

```
glm::lookAt (glm::vec3 (0.0f, 0.0f, 3.0f), glm::vec3 (0.0f, 0.0f, 0.0f), glm::vec3 (0.0f, 1.0f, 0.0f));
```

2) 뷰잉 변환

- 카메라의 위치 설정 예)

```
//--- 응용 프로그램
void drawScene ()
{
    glUseProgram (shaderProgram);
    glm::vec3 cameraPos   = glm::vec3(0.0f, 0.0f,  5.0f);          //--- 카메라 위치
    glm::vec3 cameraTarget = glm::vec3(0.0f, 0.0f, 0.0f);          //--- 카메라 바라보는 기준점
    glm::vec3 cameraUp    = glm::vec3(0.0f, 1.0f,  0.0f);          //--- 카메라 위쪽 방향
    glm::mat4 view = glm::mat4 (1.0f);

    view = glm::lookAt (cameraPos, cameraTarget, cameraUp);
    unsigned int viewLocation = glGetUniformLocation (shaderProgram, "viewTransform");   //--- 뷰잉 변환 설정
    glUniformMatrix4fv (viewLocation, 1, GL_FALSE, &view[0][0]);

    glBindVertexArray (VAO);
    glDrawArrays (GL_TRIANGLES, 0, 3);                             //--- 도형 그리기
}


//--- 버텍스 세이더
#version 330 core
layout (location = 0) in vec3 vPos;

uniform mat4 modelTransform;
uniform mat4 viewTransform;

void main()
{
        gl_Position = viewTransform * modelTransform * vec4(vPos, 1.0);
}
```

2) 뷰잉 변환

- 카메라 위치 바꾸기 예) z/Z 키를 누르면 카메라가 조금씩 뒤로/앞으로 이동

```
//--- 응용 프로그램
void drawScene ()
{
    glUseProgram (shaderProgram);
    glm::vec3 cameraPos   = glm::vec3(0.0f, 0.0f,  5.0f);          //--- 카메라 위치: 전역변수로 선언하여 사용할 수 있다.
    glm::vec3 cameraTarget = glm::vec3(0.0f, 0.0f, 0.0f);          //--- 카메라 바라보는 기준점 : 전역변수로 선언하여 사용할 수 있다.
    glm::vec3 cameraUp    = glm::vec3(0.0f, 1.0f,  0.0f);          //--- 카메라 위쪽 방향 : 전역변수로 선언하여 사용할 수 있다.
    glm::mat4 view = glm::mat4 (1.0f);

    view = glm::lookAt (cameraPos, cameraTarget, cameraUp);
    unsigned int viewLocation = glGetUniformLocation (shaderProgram, "viewTransform");          //--- 뷰잉 변환 설정
    glUniformMatrix4fv (viewLocation, 1, GL_FALSE, &view[0][0]);

    glBindVertexArray (VAO);
    glDrawArrays (GL_TRIANGLES, 0, 3);                          //--- 도형 그리기
}

void Keyboard(unsigned char key, int x, int y)
{
        if (key == 'z')                 cameraPos.z += 0.1;     //--- 더하는 값은 변경될 수 있음
        else if ( key == 'Z')           cameraPos.z -= 0.1;
        …

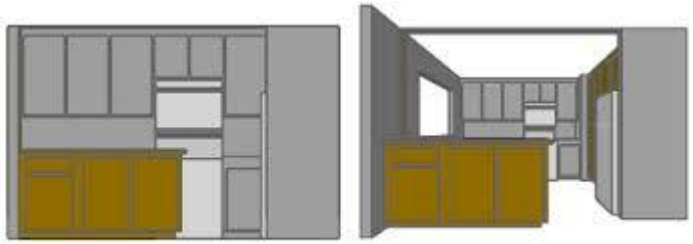
}
```

3) 투영 변환

- 투영 변환 (Projection Transformation): 3차원 장면을 2차원 화면에 투영하기 위한 공간을 설정하는 변환
  - $M_{projection}$ 행렬: 뷰 공간(View Space)의 좌표를 투영 공간(Clip Space)으로 변환하는 행렬
    - 투영 볼륨(Projection Volume): 카메라가 볼 수 있는 공간의 범위
      - 직각 투영: 직육면체 형태의 투영 볼륨
      - 원근 투영: 피라미드 형태의 절두체(Frustum)
      - 공간 설정은 대개 바뀌지 않으므로 초기화 함수에서 한 번 적용하는 것이 좋음
        - 단, **창 크기가 변경되면 종횡비(aspect ratio)에 따라 투영 행렬을 다시 계산해야 할 수 있다.**

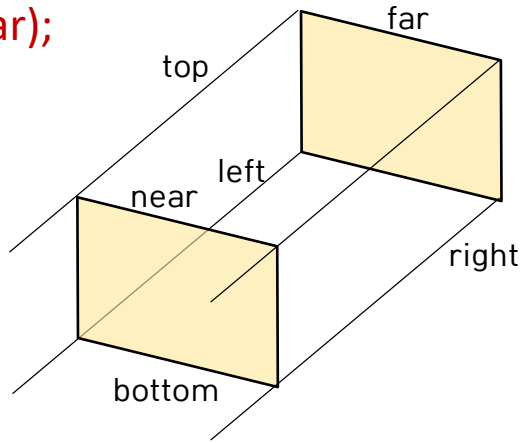
- 직각 투영
  - 투영 공간이 직육면체: orthographic projection 행렬이 적용된다.
  - 투영 볼륨 밖에 있는 부분은 **클리핑(Clipping)** 된다.
  - 3차원 객체가 화면에 **원근감 없이 투영**된다.
  - 물체의 거리에 관계없이 크기가 일정하게 투영되며 원근감이 없다.

- 원근 투영
  - 투영 공간이 피라미드 형태: perspective projection 행렬이 적용된다.
  - 투영 볼륨 밖에 있는 부분은 **클리핑(Clipping)** 된다.
  - 가까운 물체는 크게, 먼 물체는 작게 보인다.
  - 실제 사람이 보는 것과 유사한 **원근감**을 가진다.

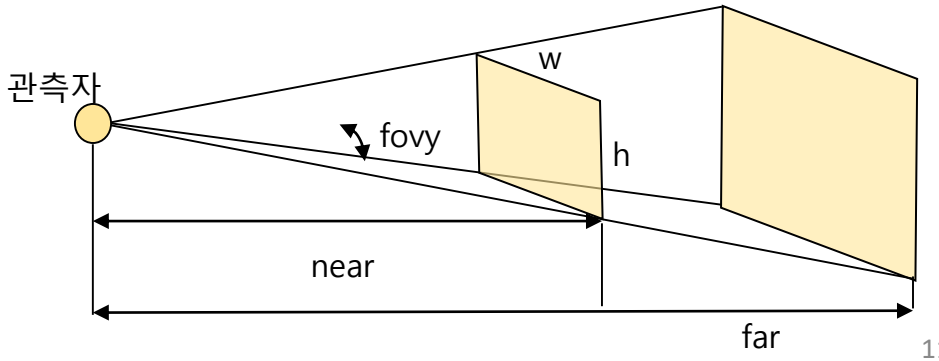


3) 투영 변환

- 직각 투영 볼륨 함수 프로토타입
  - glm::mat4 glm::ortho (float left, float right, float bottom, float top, float near, float far);
    - left, right: 투영 볼륨의 왼쪽, 오른쪽 경계
    - bottom, top: 투영 볼륨의 아래쪽과 위쪽 경계
    - near, far: 투영 볼륨의 z 범위를 지정하는 값

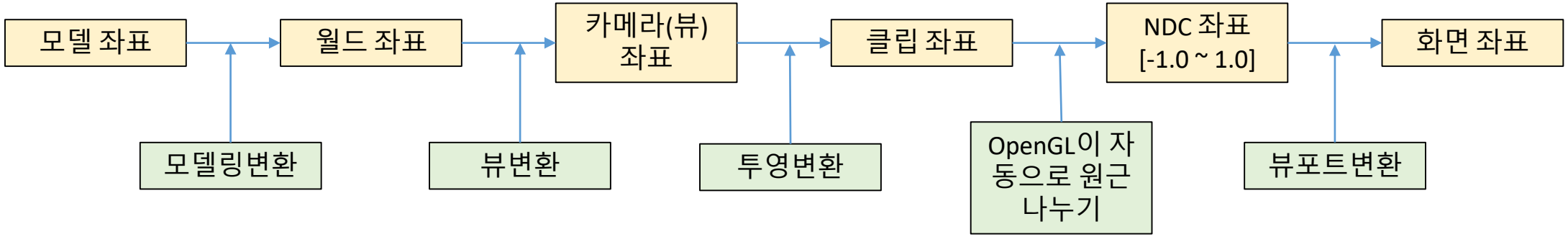


- 원근 투영 볼륨 함수 프로토타입
  - glm:: mat4 glm::perspective (float fovy, float aspect, float near, float float far);
    - fovy: 수직 방향의 시야각(Field of View, 라디언 단위)
    - aspect: 종횡비 (가로 길이를 세로 길이로 나눈 값)
      - aspect = width / height
    - near: 카메라에서 가까운 클리핑 평면까지의 거리 (항상 양수)
    - far: 카메라에서 먼 클리핑 평면까지의 거리 (항상 양수)
      - 0 < near < far



3) 투영 변환

- 투영 행렬을 적용하면
  - 좌표는 투영 공간(Clip Space)으로 변환된다.
  - 이후 OpenGL 그래픽스 파이프라인이 자동으로 원근 나누기를 수행하여 정규화 장치 좌표(NDC)로 변환한다.
  - 프로그래머가 좌표를 직접 -1~1 범위로 정규화할 필요는 없다.
  - 투영 행렬과 OpenGL 그래픽스 파이프라인이 이 과정을 처리한다.



3) 투영 변환

- 직각 투영 변환 예)

```
//--- 응용 프로그램
void drawScene ()
{
	glUseProgram (shaderProgram);
	glm::mat4 projection = glm::mat4(1.0f);
	projection = glm::ortho (-100.0f, 100.0f, -100.0f, 100.0f, -100.0f, 100.0f);		//--- 투영 공간 설정: [-100.0, 100.0]

	unsigned int projectionLocation = glGetUniformLocation (shaderProgram, "projectionTransform");		//--- 투영 변환 값 설정
	glUniformMatrix4fv (projectionLocation, 1, GL_FALSE, &projection[0][0]);

	glBindVertexArray (VAO);
	glDrawArrays (GL_TRIANGLES, 0, 3);		//--- 도형 그리기
}

//--- 버텍스 세이더
#version 330 core
layout (location = 0) in vec3 vPos;

uniform mat4 modelTransform;
uniform mat4 viewTransform;
uniform mat4 projectionTransform;

void main()
{
	gl_Position = projectionTransform * viewTransform * modelTransform * vec4(vPos, 1.0);
}
```

3) 투영 변환

- 원근 투영 변환 예)

```
//--- 응용 프로그램
void drawScene ()
{
	glUseProgram (shaderProgram);
	glm::mat4 projection = glm::mat4(1.0f);
	projection = glm::perspective (glm::radians(45.0f), 1.0f, 0.1f, 100.0f);		//--- 투영 공간 설정: fovy, aspect, near, far

	unsigned int projectionLocation = glGetUniformLocation (shaderProgram, "projectionTransform");		//--- 투영 변환 값 설정
	glUniformMatrix4fv (projectionLocation, 1, GL_FALSE, &projection[0][0]);

	glBindVertexArray (VAO);
	glDrawArrays (GL_TRIANGLES, 0, 3);		//--- 도형 그리기
}

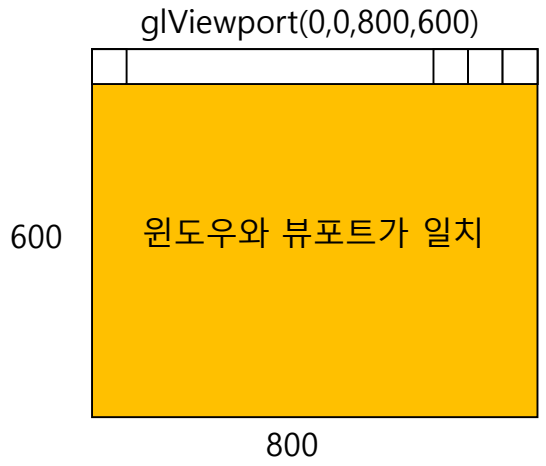
//--- 버텍스 세이더
#version 330 core
layout (location = 0) in vec3 vPos;

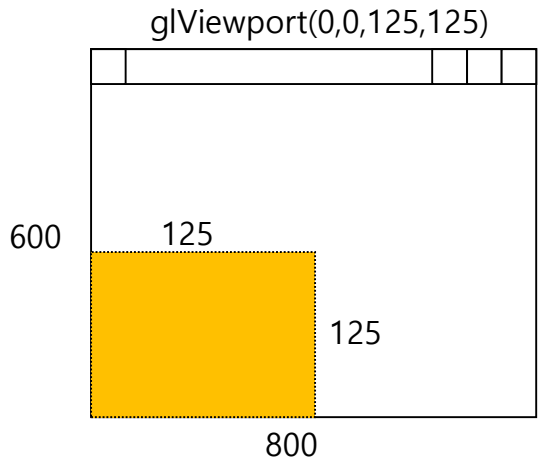
uniform mat4 modelTransform;
uniform mat4 viewTransform;
uniform mat4 projectionTransform;

void main()
{
	gl_Position = projectionTransform * viewTransform * modelTransform * vec4(vPos, 1.0);
}
```

4) 뷰포트 변환

- 뷰포트 변환 (Viewport Transformation):
  - 정규화 장치 좌표(NDC)가 실제 윈도우의 어느 영역에 출력될지를 설정
  - 화면에 최종적으로 출력될 영역을 설정
  - void **glViewport** (GLint x, GLint y, GLsizei width, GLsizei height);
    - 윈도우의 영역을 설정한다.
      - x, y: 뷰포트 사각형의 왼쪽 아래 좌표
      - width, height: 뷰포트의 너비와 높이
  - 기본적으로 뷰포트는 전체 윈도우 영역으로 설정한다.
    - OpenGL 초기화 과정에서 뷰포트를 설정한다.
    - 윈도우 크기가 변경되면 새로운 크기에 맞게 다시 설정한다.
    - GLFW에서는 일반적으로 framebuffer size callback에서 호출한다.





변환 적용하기

- 버텍스 세이더에 변환 행렬을 적용
  - 응용 프로그램에서 변환 행렬 설정
    - M_{model} 행렬

```
glm::mat4 model = glm::mat4(1.0f);
model = glm::rotate(model, glm::radians(30.0f), glm::vec3(1.0f, 0.0f, 0.0f));          //--- x축에 대하여 30도 회전
```

    - M_{view} 행렬

```
glm::mat4 view = glm::mat4(1.0f);
view = glm::translate(view, glm::vec3(0.0f, 0.0f, -3.0f));          //--- 카메라를 z축으로 3만큼 이동, 카메라와 객체 변환은 반대
```

    - $M_{projection}$ 행렬

```
glm::mat4 projection = glm::mat4(1.0f);
projection = glm::perspective(glm::radians(45.0f), (float) screenWidth / (float)screenHeight, 0.1f, 100.0f);    //--- 원근 투영 적용
```

변환 적용하기

- 버텍스 세이더에서 변환 행렬 적용

```
#version 330 core
layout (location = 0) in vec3 vPos;                 //--- 객체의 원래 버텍스 좌표값
layout (location = 1) in vec3 vColor;               //--- 객체의 색상값

uniform mat4 model;                                 //--- 모델링 변환값: 응용 프로그램에서 전달 → uniform 변수로 선언: 변수 이름“model”로 받아옴
uniform mat4 view;                                  //--- 뷰잉 변환값: 응용 프로그램에서 전달 → uniform 변수로 선언: 변수 이름“view”로 받아옴
uniform mat4 projection;                            //--- 투영 변환값: 응용 프로그램에서 전달 → uniform 변수로 선언: 변수 이름“projection”로 받아옴

out vec3 passColor;

void main()
{
    gl_Position = projection * view * model * vec4(vPos, 1.0);      //--- 변환은 ← 방향으로 적용
                                                                    //--- vPos: 객체의 원래 좌표값
    passColor = vColor;
}
```

변환 적용하기

- 응용 프로그램의 그리기 함수에서 버텍스 세이더에 uniform으로 선언되어 있는 변환 행렬 값을 전달

```
void drawScene ()
{
	glUseProgram (shaderProgram);
	glBindVertexArray (VAO);

	int modelLoc = glGetUniformLocation (shaderProgram,  "model");		//--- 버텍스 세이더에서 모델링 변환 행렬 변수값을 받아온다.
	int viewLoc = glGetUniformLocation (shaderProgram, "view");		//--- 버텍스 세이더에서 뷰잉 변환 행렬 변수값을 받아온다.
	int projLoc = glGetUniformLocation (shaderProgram,  "projection");	//--- 버텍스 세이더에서 투영 변환 행렬 변수값을 받아온다.

	//--- 모델링 변환, 뷰잉 변환, 투영 변환 행렬을 설정한 후, 버텍스 세이더에 저장한다.
	glm::mat4 mTransform = glm::mat4 (1.0f);
	mTransform = glm::rotate (mTransform, glm::radians(yAngle), glm::vec3(0.0f, 1.0, 0.0f));
	glUniformMatrix4fv (modelLoc, 1, GL_FALSE, & mTransform[0][0]);

	glm::mat4 vTransform = glm::mat4 (1.0f);
	 vTransform = glm::lookAt (cameraPos, cameraTarget, cameraUp);
	glUniformMatrix4fv (viewLoc, 1, GL_FALSE, & vTransform[0][0]);

	glm::mat4 pTransform = glm::mat4 (1.0f);
	 pTransform = glm::perspective (glm::radians(60.0f), (float)window_w / (float)window_h, 0.1f, 200.0f);
	glUniformMatrix4fv (projLoc, 1, GL_FALSE, & pTransform[0][0]);

	//--- 객체를 그린다.
	glBindVertexArray (VAO);
	glDrawArrays (GL_TRIANGLES, 0, 12);
}
```

변환 적용하기

- 뷰포트 변환 적용

```
//--- 콜백 함수 등록
glfwSetFramebufferSizeCallback(window, framebuffer_size_callback);


void framebuffer_size_callback (GLFWwindow* window, int width, int height)
{
	//--- 창 크기가 바뀌면 viewport만 바꾸는 것이 아니라 perspective projection의 aspect도 필요하면 다시 계산한다.
	glViewport(0, 0, width, height);
}
```